JERSA_TEST_FRAMEWORK · ETAPY 1 I 5

Mapa środowiska testowego

Gdzie działa sklep, gdzie działają testy i jak jedno widzi drugie.

Co jest w Dockerze?

Tylko sklep i jego baza.

Dwa kontenery: aplikacja Next.js i Postgres. Testów tam nie ma.

Gdzie uruchamiam testy?

Lokalnie, w IDE albo terminalu.

Zwykły `pytest` w venv. Debugujesz jak zawsze, z breakpointami.

Uruchomiony Docker to tylko instancja aplikacji?

Tak. To prywatna kopia sklepu.

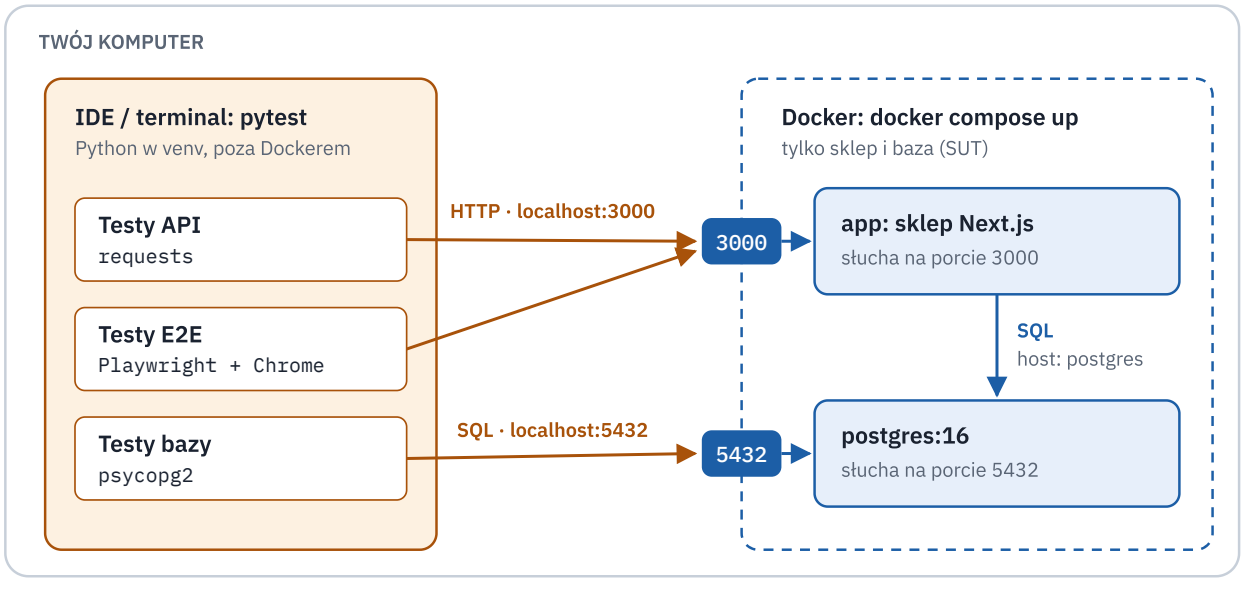
Jak mały staging na Twoim laptopie, z czystą bazą przy każdym starcie.

Czy testy zachowują się jak na stagingu?

Tak, zmienia się tylko adres.

Testy wysyłają te same żądania. Inny jest tylko `BASE_URL` w pliku `.env`.

1. Jak testy widzą sklep w Dockerze



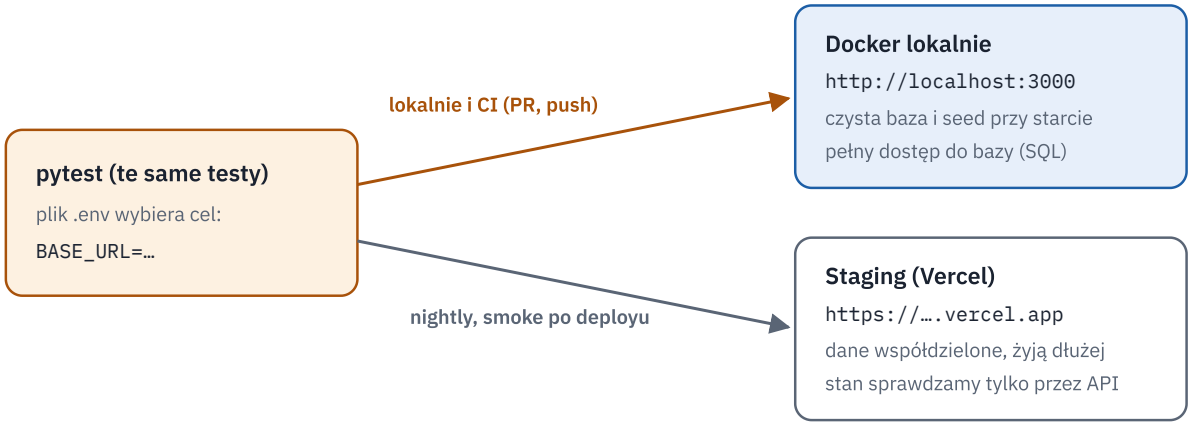
Niebieskie prostokąty na krawędzi Dockera to wystawione porty. Docker „przebija” port z kontenera na Twój komputer, więc dla testów sklep jest pod `localhost:3000`, a baza pod `localhost:5432`. Sklep i baza rozmawiają ze sobą wewnątrz Dockera, po nazwie `postgres`.

Za wystawienie portów odpowiada jedna linijka na serwis w `docker-compose.yml`:

```
# "port na laptopie : port w kontenerze"
services:
  app:
    ports: ["3000:3000"]   # sklep → http://localhost:3000
  postgres:
    ports: ["5432:5432"]   # baza → localhost:5432
```

> Bez `ports` sklep działałby w kontenerze, ale nic poza Dockerem by go nie widziało: ani przeglądarka, ani pytest.

2. Te same testy, dwa różne cele

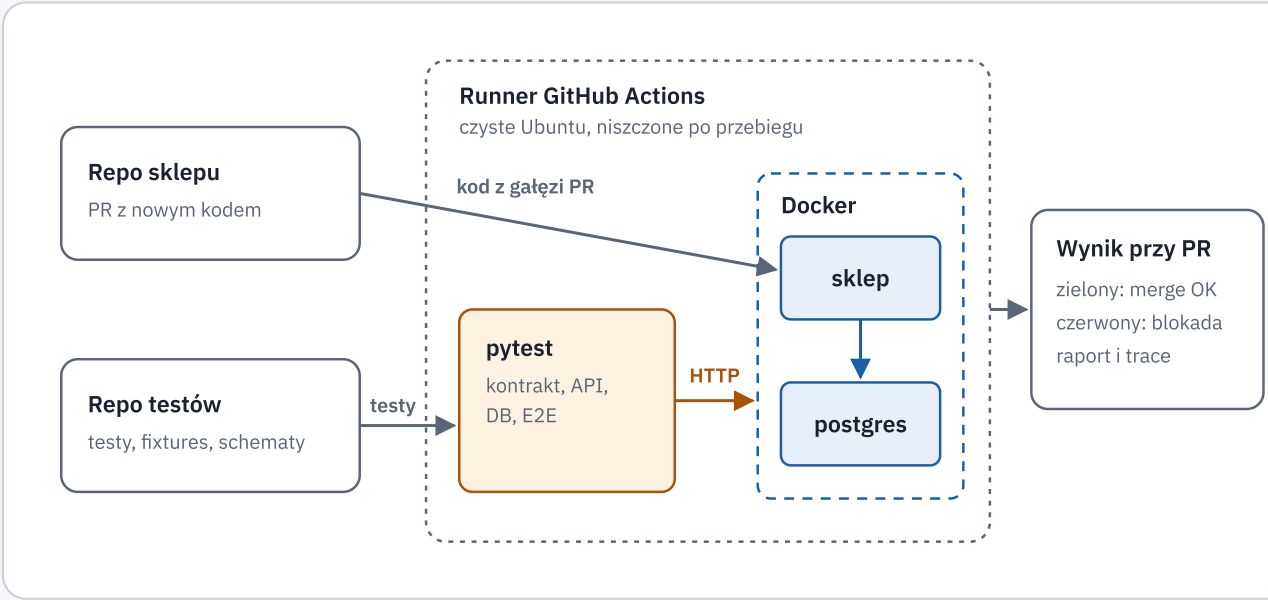


Testy nie wiedzą, czy rozmawiają z Dockerem, czy ze stagingiem. Znają tylko adres z `BASE_URL`. Zmieniasz jedną zmienną i ten sam zestaw testów idzie w inne miejsce.

	Docker lokalnie	Staging (Vercel)
Kto go stawia	Ty, komendą `docker compose up`	Vercel, po deployu sklepu
Dane na starcie	Zawsze te same (seed)	Takie, jakie zostały po poprzednich testach i ludziach
Silnik bazy	Postgres	Postgres (Neon)
Testy bazy (psycopg2)	Tak, bezpośrednio	Nie, stan sprawdzamy przez API
Co sprawdza	Czy kod sklepu działa	Czy działa wdrożona wersja w chmurze
Kiedy używamy	Lokalnie i w CI: PR sklepu, push testów	Nightly regression i smoke po deployu

3. Ten sam układ w CI

W CI „Twój komputer” zamienia się w runner: maszynę z Ubuntu, którą GitHub uruchamia na jeden przebieg i potem niszczy. Docker i pytest działają na tym samym runnerze, więc `localhost:3000` działa tak samo jak u Ciebie.



Główna bramka: każdy PR w repo sklepu stawia sklep z kodu tego PR-a i uruchamia na nim pełny zestaw testów. Błąd zostaje złapany przed merge, zanim trafi na staging.

Wyzwalacz	Wersja sklepu	Gdzie	Po co
PR w repo sklepu	gałąź PR-a	Docker na runnerze	**Główny.** Czy nowy kod sklepu niczego nie psuje, przed merge.
Push / PR w repo testów	`main`	Docker na runnerze	Czy zmienione testy działają. Zmienia się tylko jedna rzecz.
Nightly (cron, UTC)	wdrożona	staging	Pełny regression na wersji w chmurze, bez testów bazy.
Po deployu	wdrożona	staging	Szybki smoke: czy wdrożenie się udało.

> To przesunięcie testów w lewo: z „po deployu” na „przed merge”. Pełny shift-left sięga jeszcze wcześniej, do testów jednostkowych sklepu (Vitest) i uruchamiania testów lokalnie przed pushem.

4. Jeden przebieg lokalnie, krok po kroku

1. `docker compose up`: startuje Postgres, potem sklep. Sklep wgrywa schemat i seed do pustej bazy.
2. Healthcheck czeka, aż `/api/commerce/search?q=castello` zwróci CASTELLO. To dowód, że sklep i seed są gotowe.
3. Uruchamiasz `pytest` w IDE. Testy wysyłają żądania na `localhost:3000` i czytają bazę na `localhost:5432`.
4. Playwright otwiera przeglądarkę na Twoim komputerze i wchodzi na `http://localhost:3000`, jak zwykły użytkownik.
5. `docker compose down -v`: kontenery i dane znikają. Następny przebieg znowu startuje od czystego seedu.

> W CI runner robi te same kroki. Dochodzą dwa: pobranie kodu obu repozytoriów na początku i wysłanie raportu, trace i wideo jako artefaktów na końcu, także gdy testy padną.